

External agents on Front

A scoped, auditable way for third-party AI to read and act on Front

Developer Platform · PRD walkthrough

WHY NOW

Three trends outside Front

Scale

Gartner: agentic AI in 40% of enterprise apps by 2026, a third of all enterprise software by 2028.

Identity is unsolved

Non-human identities already outnumber human ones 17x to 80x+ in the enterprise, growing roughly 40% a year.

The real risk

40%+ of agentic AI projects get canceled by 2027 over unclear ROI or weak risk controls.

Gartner (2025); MCP one-year retrospective, Linux Foundation Agentic AI Foundation (2025); Veza 2026 State of Identity & Access Report.

THE PROBLEM

Agents borrow a teammate's identity, not their own

What exists

OAuth-based access with real read, write, and send scopes at the tool level.

What's missing

An identity for the agent itself, distinct from the teammate it's impersonating.

Every connection today authenticates as a specific teammate and inherits that person's exact permissions. There's no admin-managed way to grant an agent its own narrower, revocable access, and any activity record shows the teammate, not whether the agent or the person actually acted.

INPUTS AND DATA POINTS

What I'd look at first

INPUT	METRIC TRACKED	DATA SOURCE
MCP beta usage	Calls by tool; scope mix; denial and error rate	MCP server logs, API gateway
Size of the "growing priority"	Agent-connection requests per quarter	Support tickets, CRM, partner inbound
Support rep baseline	First-response time; conversations per rep per day	Front's own analytics
Security precedent	Non-human identities today, and how each is scoped and revoked	Auth service inventory, security team
Interviews	Top jobs and blockers (8-10 interviews)	Customer success, partner managers
DPA coverage	Share of contracts covering an agent; days to clear	Legal contract repository

WHO THIS IS FOR

Three jobs, one platform decision

Customer (Instructure)

When conversation volume outgrows headcount, let an agent triage and draft in my workspace, so I hold response times without hiring.

Partner (Aircall)

When I build a Front integration, use one scope model any admin can grant, so I ship once instead of adapting per customer.

Third-party client (Claude Desktop)

When a user points me at any Front workspace, connect the same way every time, so I need no bespoke integration per customer.

Real, public examples: a Front customer story, a named Front Integration Partner, and a generic MCP client that needs no partnership to connect at all.

HIGHEST-LEVERAGE USE CASES

Read, draft, triage, trigger

USE CASE	RISK	WHY IT MATTERS
Read context	Low	The on-ramp for everything else the agent does
Draft a reply	Low, human sends	Highest leverage per unit of risk
Triage / classify	Medium, recoverable	Value compounds at volume
Trigger a defined workflow	Medium, scoped	Reuses automation Front already trusts

MVP

What ships first

Agent connection as a named, scoped principal, not a generic token

MCP tools gated by connection scope, not an impersonated role

Human-in-the-loop send, a structural gate, not a UI convention

Audit log of every scoped action: connection, scope, target, time

One-click revocation, effective immediately

No autonomous send, no self-expanding scopes, no marketplace, in v1

ROADMAP

Draft-and-approve first, autonomy earned later

PHASE	DURATION	SCOPE
0	1-2 wks	Scope taxonomy and data model with security and infra
1	4-6 wks	Connect/consent UI, audit log, read + draft tools. Closed beta.
2	3-4 wks	Tag/assign and workflow-trigger tools. Open beta, self-serve.
3	GA	Decide v1.1 auto-send opt-in on measured trust, not a date

RISKS

What could go wrong, and the mitigation

RISK	MITIGATION
A scoped agent is still a meaningful blast radius	Team/inbox-level default scopes, per-connection rate limits
Support reps ignore drafts, adoption stalls	Measure edit-distance and send-rate in beta; make accept faster than writing
MCP auth patterns still stabilizing industry-wide	Version the server interface from day one, track the spec's auth working group
DPA's may not contemplate a third-party agent	Legal and compliance in Phase 0; developer data-handling attestation

OUT OF SCOPE, AND NON-GOALS

What's deferred, and what's permanent

Not now (deferred)

Autonomous send

Partner marketplace or directory

SDKs beyond raw MCP tool calls

Not ever (non-goals)

Self-expanding scopes

Cross-workspace or cross-tenant access

A non-MCP transport

Each deferred item has a real revisit trigger tied to beta data, not a fixed date. Non-goals would undermine the safety model itself, not just add scope to it.

SUCCESS METRICS

Four, matching the brief's own categories

ADOPTION

30% of API-plan workspaces create a connection within 90 days of GA.

TIME TO FIRST INTEGRATION

Median under 30 minutes, credentials to first sandbox tool call.

DEVELOPER SATISFACTION

8/10+ on scope-model clarity in the post-beta survey.

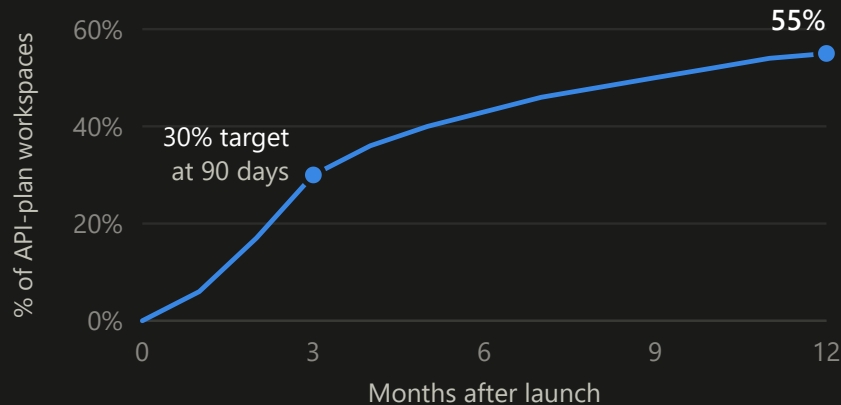
RELIABILITY

99.9% MCP server availability, P95 tool-call latency under 500ms.

Adoption first, revenue later

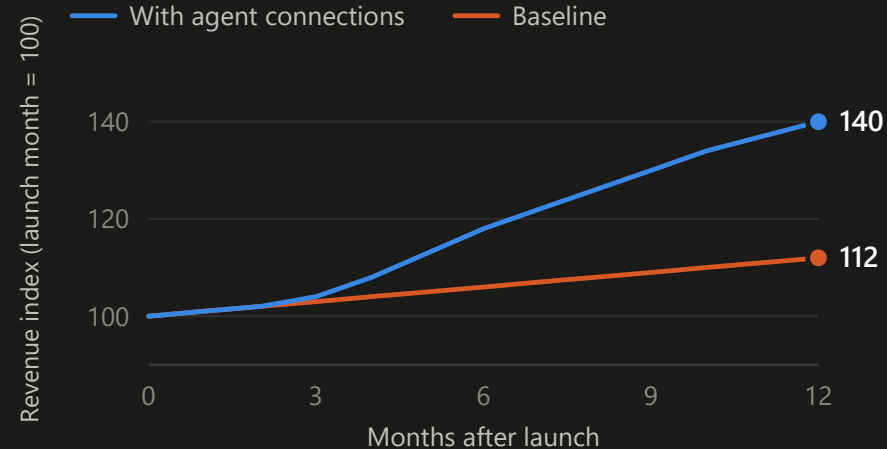
Adoption: workspaces with an agent connection

Share of API-plan workspaces with at least one connection



Revenue from existing customers

Indexed: revenue in the launch month = 100

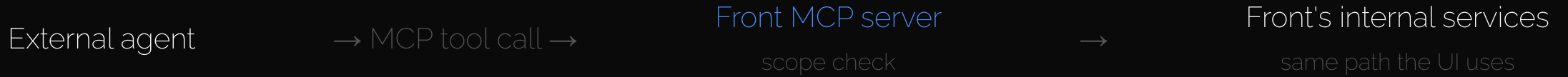


Placeholder numbers, not a forecast. Revenue starts rising about 3 months after adoption. Replace with beta data.

Hypothesis: customers that connect an agent handle more conversations per Support rep, so they buy more seats and make more API calls. Their spending rises about 3 months after they connect, so the revenue line climbs after the adoption line. An index of 140 means 40% more revenue than in the launch month. All numbers are placeholders until beta data replaces them.

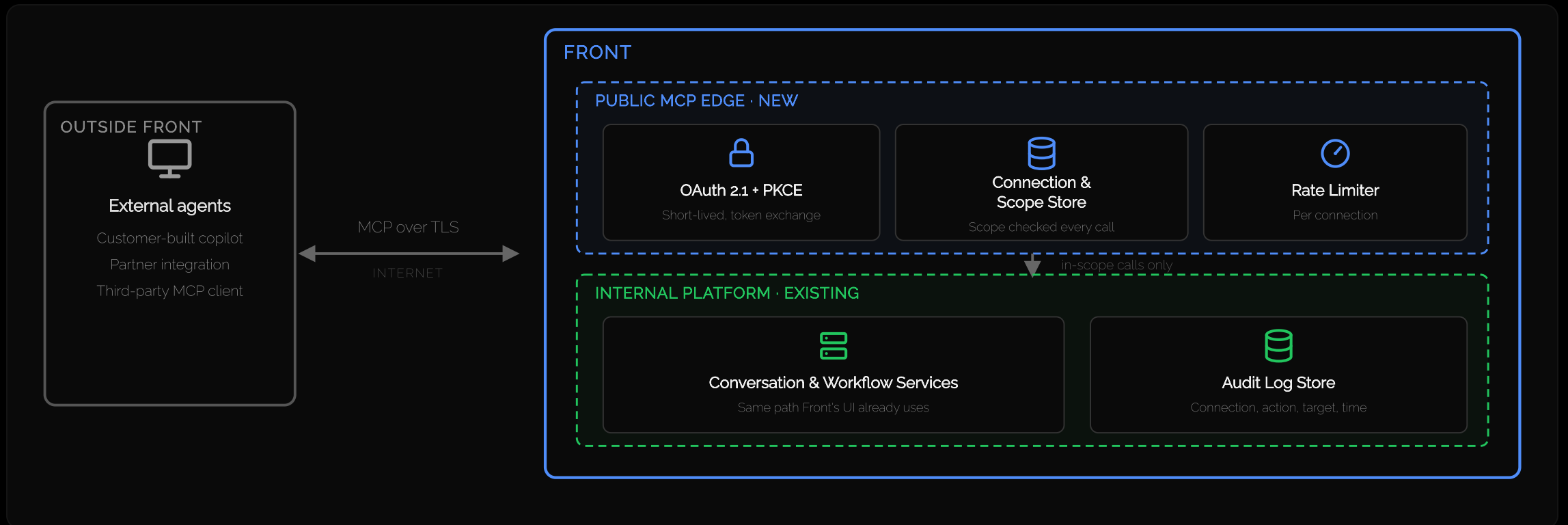
HOW IT COMMUNICATES

MCP as the door, not a new house



Writes go through the same command/event pipeline Front already uses for its own automation. Nothing new at the domain level, only a new, scoped entry point into it. [SendReply](#) is the one command no agent credential can ever reach, see the event model.

Three trust zones, one path



The Connection & Scope Store, rate limiter, and audit log are the genuinely new pieces. Everything inside the internal platform boundary already runs today, and [SendReply](#) there is issued by a Support rep, never by an agent credential.

Every scoped call, the same six steps

STEP	CONTROL
Agent calls an MCP tool over TLS	Short-lived, token-exchange credential, not a long-lived key
Edge looks up the connection's stored scopes	Server-side only; the caller's claim is never trusted
Scope check gates the call	Zero tolerance for scope escapes; denials are logged too
In-scope call reaches Front's existing service	No second execution path to maintain or drift from
Service performs the action under its own rules	e.g. draft only, a workflow already approved
Every outcome writes one audit entry	Connection, action, target, time; retained 1+ year

P95 latency: under 500ms read, under 800ms write. Rate limits by tier: roughly 120/30/20 calls per minute. Revocation reaches every future call in under a minute.

TECHNICAL TRADE-OFFS

What I'd work through with engineering

Coupling vs. gateway: fast to ship vs. safe to evolve independently

Sync vs. async tool calls: simple today vs. what long-running actions need

Fine-grained vs. bundled scopes: least privilege vs. easy to reason about

Per-connection vs. per-workspace rate limits: predictable vs. infra-safe

THE HARDEST ASSUMPTION

Human review is enough to de-risk launch

If customers push for auto-send in week one because draft-and-approve doesn't save enough time, the fix is that the permission and audit layer, the actual safety mechanism, was already built robust enough to support it. Extending to v1.1 becomes a scope change, not a re-architecture.